Bridging Simulation and Experiment with Deep Neural Networks for Estimating Protein-Protein Binding Affinity

Rohan Devereux

The thesis is submitted to University College Dublin
in part fulfilment of the requirements for the degree of
BSc Applied and Computational Mathematics

School of Mathematics and Statistics
and
School of Physics
University College Dublin

Supervisor: Dr. Donal McKernan (UCD)
Dr. Jony Castagna (STFC CECAM-UK-DARESBURY)

April 2026

Abstract

Protein–protein binding affinity measures how strongly two proteins associate and matters for protein design, therapeutic targeting, and drug discovery. Most fast predictors estimate affinity from a single bound structure, even though binding is an ensemble property and proteins can adopt more than one relevant arrangement. This study tests whether short explicit-solvent molecular dynamics (MD) can add useful information to a residue-graph affinity predictor.

Starting from a curated wild-type subset of PPB-Affinity, each retained complex was prepared for MD, simulated in OpenMM, converted to a residue graph, and enriched with trajectory-derived summary features. The final controlled comparison used 1437 complexes and 52 models, combining full or interface-restricted graphs, static or static-plus-dynamic features, and two model families: a masked GraphVAE with affinity predicted from exported latent means by ridge regression, and a direct supervised graph regressor. All models were evaluated over 10 outer held-out fits.

The strongest model was the direct supervised baseline on the full static graph, with mean test RMSE 2.3379 ± 0.1067, Pearson correlation 0.5064 ± 0.0567, and $R^2 = 0.2146 \pm 0.0891$. The full graph outperformed the interface-restricted graph in all 24 matched GraphVAE comparisons, and static features outperformed static-plus-dynamic features in 11 of 12 matched GraphVAE comparisons under the static-only reconstruction target.

On this benchmark, the static whole-complex representation remained strongest. The results suggest that 1.2 ns trajectories captured little new affinity-relevant motion, while coarse trajectory summaries made any weak added signal difficult to use reliably.

Acknowledgments

I'd like to thank Dr. Donal MacKernan and Dr. Jony Castagna for supervising this thesis and providing guidance and advice. I'd also like to thank Shrinath Kumar for helping with molecular dynamics simulations using OpenMM. Finally, I'd like to thank UCD for allowing access to the Sonic HPC, without which the computationally intensive simulations used in this project would not have been possible.

Contents

1 Introduction **1**

2 Literature Review **3**

2.1 Protein–protein benchmark 3
2.2 Protein–protein methods 4
2.3 Protein–ligand comparison 4
2.4 Generalisation and evaluation 5
2.5 Variational models 6
2.6 Study position 6

3 Methods **7**

3.1 Dataset and target 7
3.2 Molecular-dynamics preparation and simulation 8
3.3 Residue-graph construction and features 8
3.4 Graph views, analysis subset, and preprocessing 10
3.5 Models 10
3.5.1 Direct supervised baseline 11
3.5.2 Masked GraphVAE 12
3.5.3 Latent-space ridge regression 13
3.6 Repeated outer evaluation 13

4 Results **15**

4.1 Headline models 15
4.2 Factor comparisons 15
4.3 Stability under repeated evaluation 16
4.4 Family-level train, validation, and test behaviour 17
4.5 Performance by interaction class 17
4.6 Summary 18

5 Discussion and Conclusions **20**

5.1 Main findings 20
5.2 Limitations 21

5.3 Future work 22
5.4 Conclusion 22

A Archived Approaches and Project Evolution **23**
A.1 Correction learning around fast affinity baselines 23
A.1.1 PRODIGY 23
A.1.2 MM/GBSA 24
A.2 The Cα contact-map approach 24
A.3 PPB-Affinity interface-attention augmentation 25

B Complete Model Matrix **27**

C Repository Layout **30**

List of Figures

3.1 Model structure used in the comparison. The two model families use the same encoder architecture: linear node and edge projections, three GINE layers, and mean-plus-max graph pooling. The supervised baseline predicts ΔG directly from the pooled graph embedding. The GraphVAE maps the pooled graph embedding to μ and $\log \sigma^2$, uses sampled latent vectors for masked reconstruction, uses the latent mean for the optional affinity head during training, and reports affinity from ridge regression on exported latent means. 11

4.1 Main empirical picture of the 52-model comparison. Left: top of the model ranking by mean test RMSE with one-standard-deviation error bars over the 10 outer fits. Right: matched GraphVAE factor comparisons, reported as the mean test-RMSE advantage for the first-named option. 17

4.2 Subgroup performance of the three headline models. Points show the mean outer-test metric and error bars show one standard deviation across the 10 outer fits. The model ordering is stable across subgroups, but uncertainty is visibly larger for the 105-complex TCR-pMHC subset. 19

List of Tables

2.1 Representative reported results from the protein–protein affinity literature. Reported correlations are taken from the original papers; the datasets differ across rows. 5

3.1 Composition of the full curated wild-type table and the retained 1437-complex matched subset. The retained set is enriched for antibody–antigen complexes and has a slightly more negative mean binding free energy. . . . 10

4.1 Best-performing models in the 52-model comparison. Reported RMSE, Pearson r, and R^2 values are mean $\pm$ standard deviation over the 10 outer fits. GraphVAE metrics are those obtained after ridge regression on latent means. 16

4.2 Matched GraphVAE comparisons. A positive RMSE gain means the first-named option has lower mean test RMSE. 16

4.3 Family-level mean train, validation, and test behaviour. Here $g_r = r_{\text{train}} - r_{\text{test}}$. Each row is averaged over all models in that family. 18

4.4 Pooled outer-test subgroup metrics for the best overall model (full static baseline), the best GraphVAE (full static semi-supervised GraphVAE, latent dimension 32), and the best interface model (interface static baseline). 18

A.1 Saved subset-specific PRODIGY results from archived curated datasets. The TCR-pMHC subset was especially weak, while the small antibody–antigen subset was somewhat better. 24

A.2 Representative archived PRODIGY-based regression runs on the TCR subset. The direct model is the learned latent-space regressor, while the corrected model tries to learn an adjustment to PRODIGY. 24

A.3 MM/GBSA partial-result summary from the archived approach. The estimates were far outside the scale of experimental protein–protein binding free energies, so this route was dropped as a quantitative baseline. 25

A.4 Representative Cα contact-map regression runs on the archived TCR subset. Some runs reduced RMSE relative to raw PRODIGY, but the results were unstable and often had weak or negative test correlation. 25

A.5 Core ideas in the archived PPB-Affinity augmentation branch. 26

B.1 Complete 52-model evaluation matrix, sorted by mean test RMSE. Metrics are means over 10 outer held-out fits. 28

Chapter 1

Introduction

Protein–protein binding affinity prediction asks how strongly two proteins bind. In benchmark datasets for this task, the target is a thermodynamic quantity such as dissociation constant K_D or binding free energy ΔG, while the input is usually a single three-dimensional complex structure [1]. Accurate affinity prediction is important for protein design, therapeutic targeting, and drug discovery [2, 3]. Experimental measurements provide the ground truth, but they are too slow and expensive for large exploratory searches. Physics-based free-energy calculations aim more directly at binding free energy, but the more faithful versions are also expensive [4, 5]. That is why many practical machine-learning predictors try to estimate affinity quickly from structural input alone.

One difficulty is that binding affinity is an ensemble property: it depends on the range of shapes and relative arrangements the proteins sample across an ensemble, beyond any one frozen structure. Proteins can bend, shift, and sometimes slide against each other as they bind [6], yet many predictors still see only one static structure. Recent protein–protein models also differ in how much structure they keep: some focus tightly on the interface, others use whole-complex descriptors, and hybrid models combine complete and interface-aware information [1, 7–9]. A related question, explored more extensively in protein–ligand affinity prediction, is whether MD can add useful information beyond a single bound structure [10, 11]. This study brings those two questions together in one controlled protein–protein comparison. Here, a residue graph treats amino-acid residues as nodes and residue–residue proximity or contact as edges.

The analysis uses a curated wild-type, that is, non-mutant, subset of PPB-Affinity [1]. The final comparison uses a matched subset of 1437 complexes from a curated table of 2818. Each retained complex went through the same preparation and short-MD procedure before conversion into a post-preparation residue graph with DeepRank2 and enrichment with trajectory-derived summaries. Those graph representations were then used in two model paths. One path trained a masked GraphVAE and predicted affinity from the exported latent means with ridge regression; the other used a direct supervised graph regressor. The comparison tests whether an affordable MD augmentation step can

preserve enough extra information to beat a static baseline.

The comparison asks four questions. Is a full residue graph better than an interface-restricted representation? Do MD-derived summaries improve prediction over static graph features alone? Does a masked GraphVAE produce a better predictive representation than a direct supervised graph regressor with the same encoder architecture? How stable are those answers under repeated held-out evaluation?

Chapter 2 places the work in the literature on protein–protein affinity prediction, molecular simulation, and learned structural representations. Chapter 3 describes the methods. Chapter 4 reports the 52-model comparison. Chapter 5 gives the main conclusions. The appendices cover archived approaches, the complete model table, and the repository layout.

The implementation is organised in `https://github.com/RohanDevereux/Bridging`. Appendix A records the alternative approaches explored before the final study design was settled.

Chapter 2

Literature Review

2.1 Protein–protein benchmark

Protein–ligand benchmarking has had a shared structural resource since PDBbind [12], whereas curated protein–protein datasets for absolute affinity remained much smaller for many years [13]. Protein–protein affinity prediction has therefore developed with less data and with more variation between datasets. PPB-Affinity was introduced to improve that situation by building a larger benchmark specifically for direct protein–protein affinity prediction [1]. Reviews of the field repeatedly point to three practical difficulties: limited affinity data, mixed interaction classes, and proteins changing shape as they bind [2, 5]. Protein–protein affinity prediction is therefore hard for statistical, biological, and structural reasons at the same time.

That last point makes single-structure scoring especially restrictive. In the Kastritis benchmark, only a minority of complexes were close to rigid-body association, and large or moderate interface rearrangements were common [13]. Many protein pairs bind through bending, shifting, or interface reorganisation instead of simple rigid docking. A useful predictor therefore has to cope with both shape changes within a complex and structural differences across complexes.

PPB-Affinity also shows how heterogeneous this prediction task is. Its benchmark model reached Pearson $r = 0.701$ on the full test set, but subgroup performance was weaker for both antibody–antigen and T-cell receptor–peptide major histocompatibility complex (TCR-pMHC) complexes. The authors linked the TCR-pMHC difficulty to strong interface similarity within that class and noted that both subgroups contain a higher proportion of mutant samples than the full dataset [1]. Later antibody-specific models make the same point from another angle: Yang et al. argued that general protein–protein affinity predictors often transfer poorly to antibody–antigen complexes and proposed specialised regressors instead [14]. Subgroup performance is therefore central in this field. Different interaction classes combine different affinity ranges, interface similarities,

and mutation patterns, so they can reward different representations.

2.2 Protein–protein methods

Umbrella sampling and alchemical free-energy methods aim more directly at binding free energy, but they are expensive and sensitive to protocol design [4, 15]. Most of that literature is protein–ligand focused. For protein–protein systems, the same calculations are usually harder because the complexes are larger and more flexible [5]. MM/PBSA and MM/GBSA are cheaper endpoint approximations [16], but they still depend strongly on setup choices and require careful interpretation. At the other end of the spectrum, fast empirical models use compact structural summaries. PRODIGY predicts affinity from interface contact counts and non-interacting surface descriptors [17], while Moal et al. used engineered whole-complex descriptors [7].

Current learned protein–protein models make different choices about how much structure to keep and how tightly to focus on the interface. PRODIGY and the PPB-Affinity benchmark model are explicitly interface-focused [1, 17]. Moal et al. and PPI-Affinity instead build descriptor sets from the complex as a whole rather than from a fixed interface crop [7, 8]. PPI-Graphomer sits between those extremes: it builds an interface-aware module, but the final predictor combines that interface representation with complete sequence and structure information [9]. Across these models, the central trade-off involves scope as well as architecture: interface-focused methods trade away global context for a cleaner local signal, while whole-complex and hybrid methods keep more structural background. That trade-off is central here because the final comparison tests it directly under matched training and evaluation conditions.

2.3 Protein–ligand comparison

Protein–ligand affinity prediction differs from protein–protein affinity prediction in ways that matter here. In protein–ligand work, the ligand is usually much smaller, the interaction is more tightly localised to one binding pocket, and larger shared resources such as PDBbind have been available for much longer [12]. Protein–protein systems are larger, more heterogeneous, and often require the model to decide how much of two extended molecular surfaces should be retained [2, 5]. That difference has practical consequences. In protein–ligand prediction, pocket-restricted voxel or graph representations are often a natural choice. In protein–protein prediction, the same design can discard context that still contributes to binding strength.

Even so, the protein–ligand literature is useful as a guide to promising ideas. Pafnucy showed that a fixed local 3D representation could support successful affinity prediction when the binding problem is tightly localised [18]. More recent work has gone further

Domain	Method	Representation / scope	Model class	Reported result
Protein–protein	PRODIGY [8, 17]	Interfacial contacts and non-interacting surface descriptors; interface-only	Empirical regression	Original paper: $R = -0.73$ on a cleaned benchmark in the paper's sign convention; later PPI-Affinity re-evaluation: 79-complex benchmark subset $r = 0.74$, MAE 1.4 kcal mol^{-1}; external 90-complex set $r = 0.31$, MAE 2.5 kcal mol^{-1}
Protein–protein	Moal et al. [7]	Engineered descriptor set over diverse complexes; whole-complex with extra terms for flexibility and rearrangement	Ensemble of regressors	Diverse 137-complex benchmark: $r = 0.73$, RMSE 2.06 kcal mol^{-1}
Protein–protein	PPI-Affinity [8]	Structure-derived descriptor vector; whole-complex descriptors	Ensemble regressor	Vangone–Bonvin benchmark: $r = 0.62$, MAE 1.8 kcal mol^{-1}; external 90-complex hold-out: $r = 0.50$, MAE 1.8 kcal mol^{-1}
Protein–protein	PPB-Affinity benchmark model [1]	128 interface residues and pair geometry; interface-restricted	Geometry-aware deep network	Whole-dataset benchmark: Pearson $r = 0.701$, Spearman $\rho = 0.691$
Protein–protein	PPI-Graphomer [9]	Complete sequence and structure features combined with an interface-aware module; hybrid complete-plus-interface	Graph transformer	Two external test sets: Pearson $r = 0.641$ and 0.625

Table 2.1: Representative reported results from the protein–protein affinity literature. Reported correlations are taken from the original papers; the datasets differ across rows.

by using MD more directly. Dynaformer argues that static structures can miss affinity-relevant information present in conformational ensembles, while Timenucy and Videonucy learn from frame sequences rather than from simple trajectory averages [10, 11]. These papers still guide the protein–protein question indirectly. They point to two ideas that matter here: local structural representations can work well when the interaction is strongly localised, and dynamic information is more likely to help when the model retains richer trajectory information than a small set of summary statistics.

2.4 Generalisation and evaluation

Reported accuracy in this area depends strongly on how train and test examples are separated. PPI-Affinity reported that, in its own re-evaluation, PRODIGY reached $r = 0.74$ on a 79-complex benchmark subset but only $r = 0.31$ on a separate 90-complex hold-out set [8]. Moal et al. had already tried to reduce the same problem by removing homologous complexes, so evaluation stayed clear of near-duplicate training examples [7]. In protein–ligand scoring, Graber et al. later showed that train–test leakage between

PDBbind and CASF had inflated benchmark performance, and that stricter splits caused clear drops for previously strong models [19]. Split design is therefore part of the scientific interpretation: reported gains can come from a stronger model, an easier split, or both. That issue is relevant here because the repeated splits in this thesis are by complex identifier only, leaving homology and structural clustering unconstrained.

2.5 Variational models

Variational autoencoders were introduced as latent-variable models that combine reconstruction with a regularised latent distribution [20]. Variational graph autoencoders brought the same idea to graph data [21], and later GraphVAE-style models made graph generation more explicit [22]. In molecular machine learning, related latent-space VAEs were already being used successfully for small molecules [23]. Here the point is narrower than full graph generation: the GraphVAE is used to learn a compact residue-graph representation and then test whether that representation carries useful affinity signal.

There is also a simulation-side motivation. Tian et al. showed that a VAE trained on protein MD trajectories could learn a latent space that captured physically plausible structural variation [24]. That supports the idea that a latent model may be useful when structure and MD features are high-dimensional and only part of that information is relevant to binding strength.

2.6 Study position

This study sits between fast single-structure scoring and much heavier free-energy calculation [4, 5]. It uses short explicit-solvent MD to build extra residue-graph features and asks whether a lighter MD augmentation step, motivated partly by recent protein–ligand work, can improve direct protein–protein affinity prediction [10, 11]. It therefore asks two linked questions: can very short MD add enough new affinity signal beyond a bound structure to matter, and if that signal is weak, can the same short-MD setup preserve it well enough to help?

Chapter 3

Methods

3.1 Dataset and target

All experiments used a curated wild-type subset of PPB-Affinity [1]. Wild-type here means non-mutant complexes. For consistency, each complex was stored with one ordered assignment of the two binding partners, labelled ligand and receptor. Curation retained one ordered ligand–receptor chain assignment per complex, removed duplicate entries, and excluded mutant complexes. This keeps the task focused on direct absolute-affinity prediction, distinct from mutation-effect prediction of the kind benchmarked in datasets such as SKEMPI [25]. A complex was kept only if a Protein Data Bank structure, an interpretable ligand/receptor chain definition, and an affinity measurement sufficient to derive a binding free energy were available. The resulting curated table contained 2818 complexes.

The interaction classes in this curated table were 199 T-cell receptor–peptide major histocompatibility complex (TCR-pMHC) complexes, 437 antibody–antigen complexes, and 2182 other or unannotated complexes. PPB-Affinity reports different affinity distributions and weaker benchmark performance for the TCR-pMHC and antibody–antigen subsets than for the full test set [1]. The dataset is therefore heterogeneous, and subgroup composition has to be tracked throughout the study.

The regression target was the standard Gibbs free energy of binding, ΔG, in kcal mol^{-1}. In this curated table, the affinity column used in the study was the reported dissociation constant K_D. All targets were therefore computed as

$$\Delta G = RT \ln K_D, \tag{3.1}$$

with $R = 0.00198720425864083$ kcal mol^{-1} K^{-1} and the processed temperature column `Temp_K`. This column had already been standardised during preprocessing; where source temperatures were absent, it was set to 298.15 K.

3.2 Molecular-dynamics preparation and simulation

Complexes were prepared from PDB coordinates with PDBFixer and the OpenMM application layer [26, 27]. The same short-MD procedure was applied to every complex: extract the two partner chains, clean the experimental structure, choose hydrogen and charge states, add water and salt, and then run a short simulation. The annotated ligand and receptor chains were extracted first. Missing residues were detected but not rebuilt. Nonstandard residues were converted when a standard mapping was available, heterogens were removed, missing heavy atoms (that is, non-hydrogen atoms) were added, and hydrogens were assigned before solvation. This uniform preparation made it possible to process the study set without per-complex manual tuning.

Prepared complexes were solvated with 1.0 nm padding and ionic strength 0.15 M. In the implementation, OpenMM loaded the XML files `amber14-all.xml` and `amber14/tip3pfb.xml`, corresponding to Amber protein parameters together with the TIP3P-FB water model [28, 29]. The processed PPB-Affinity table used here had no pH column, so protonation, that is, assignment of hydrogen and charge states, was carried out at pH 7.0 for every complex.

All simulations were run in OpenMM [27] with particle mesh Ewald (PME) electrostatics, a 1.0 nm real-space cutoff, constraints on bonds involving hydrogen, rigid water, a Monte Carlo barostat at 1 atm, a Langevin middle integrator with friction 1 ps^{-1}, and a 2 fs timestep. Each system was minimised for up to 1000 iterations, equilibrated for 100,000 steps, and then simulated for 500,000 further steps. Coordinate reporters were written every 5000 steps. Because the reporters were attached before equilibration, each complex contributed 120 saved frames over 1.2 ns in total. The 1.2 ns length was set by the available compute budget and kept the same across complexes.

3.3 Residue-graph construction and features

Residue graphs were built with DeepRank2 [30] from the post-preparation protein topology written after preparation. The same post-prep graph was then used for both the S and SD modes so that the comparison isolated the effect of the added trajectory summaries without also changing the structure source. The query radius of 10^6 Å was used to retain the entire prepared complex instead of trimming the graph to a local neighbourhood. DeepRank2 supplied the basic residue and edge descriptors for residue identity, contact geometry, solvent exposure, and surface area.

The graph inputs were grouped as follows. Static features were present in both model modes. Dynamic features were added only in the SD mode. Because the underlying post-prep graph is shared, the incremental difference in SD is the added trajectory-derived information. Dynamic information entered as residue- and edge-level summary features

on a fixed graph, with time order left implicit in those summaries.

- **Static node features:** DeepRank2 per-residue geometric and exposure descriptors (buried surface area, solvent-accessible surface area, residue depth, and three half-sphere exposure measures, which count neighbouring residues in different local directions); counts of nearby residues within 8 Å on the same chain and on the partner chain; residue identity encoded by a 20-way amino-acid one-hot vector, broad residue-class indicators (non-polar, polar, charged), physicochemical scalars (side-chain size, net charge, hydrophobicity, α-helix propensity, β-sheet propensity), and N- and C-terminus flags.

- **Static edge features:** residue–residue distance, same-chain indicator, and DeepRank2 electrostatic and van der Waals edge terms.

- **Dynamic node features:** hydration summaries, alpha-carbon (Cα) flexibility, backbone dihedral-angle summaries, and approximate inter-chain force summaries.

- **Dynamic edge features:** dynamic contact frequency together with the mean and standard deviation of the closest heavy-atom distance over the saved trajectory.

Dynamic node and edge features were computed from the saved trajectories with MDTraj [31]. Hydration was measured from the full solvated trajectory by counting unique water oxygen residues within 5 Å of any heavy atom of a residue. The stored hydration features were $\log(1 + \bar{w}_i)$, where $\bar{w}_i$ is the mean framewise water count for residue i, and the standard deviation of the framewise water counts. The logarithm controls the right tail from highly solvent-exposed residues while preserving zero exactly. Alpha-carbon root-mean-square fluctuation (Cα RMSF) was computed after least-squares alignment. Backbone torsions were summarised by frame-averaged values of $\sin\phi$, $\cos\phi$, $\sin\psi$, and $\cos\psi$, where ϕ and ψ are the two standard peptide-backbone dihedral angles describing rotation about the N–Cα and Cα–C bonds.

Dynamic edge features were computed only on residue pairs already connected in the static graph. For each edge, the closest heavy-atom distance was evaluated in every saved frame. The dynamic contact frequency was the fraction of frames with closest-heavy distance at most 8 Å. The same per-frame distances were also summarised by their mean and standard deviation.

Each residue also received coarse inter-chain nonbonded interaction summaries. A protein-only OpenMM analysis system was rebuilt from the saved protein topology using the same force-field files as the original simulation. The original nonbonded parameters were copied into separate Coulomb and Lennard–Jones `CustomNonbondedForce` objects in OpenMM [27] and restricted to ligand-atom versus receptor-atom interaction groups. Here Coulomb and Lennard–Jones mean electrostatic and van der Waals components

Dataset	n	Other	Antibody–antigen	TCR-pMHC	Mean ΔG
Curated wild-type table	2818	2182 (77.4%)	437 (15.5%)	199 (7.1%)	-9.88 ± 2.57
Matched final subset	1437	966 (67.2%)	366 (25.5%)	105 (7.3%)	-10.12 ± 2.65

Table 3.1: Composition of the full curated wild-type table and the retained 1437-complex matched subset. The retained set is enriched for antibody–antigen complexes and has a slightly more negative mean binding free energy.

respectively. For each saved frame, per-atom forces were accumulated to the residue level and reduced to Euclidean norms. The stored features were the framewise means and standard deviations of those residue-level norms for the two channels. These should be read as approximate residue-level interaction proxies.

3.4 Graph views, analysis subset, and preprocessing

Two graph views were evaluated. The *full* view retained the complete residue graph. The *interface-restricted* view retained residues near the closest ligand-residue/receptor-residue pair identified from mapped trajectory Cα coordinates, under a cap of 128 residues. The full view keeps the whole complex, whereas the interface view keeps only residues near the binding site. The 128-residue cap follows the fixed interface-window design used in the PPB-Affinity benchmark model [1].

The full MD-backed comparison used 1437 of the 2818 curated complexes. MD simulation, feature extraction, and repeated model evaluation were restricted to this smaller analysis subset because carrying out all three steps for every curated complex was beyond the available budget. The final paired full/interface comparison used complexes with both graph views and the full MD-derived feature set available. The retained set contained 105 TCR-pMHC, 366 antibody–antigen, and 966 other complexes. Table 3.1 compares the retained set with the full curated table.

Feature scaling was fit separately inside each outer split and only on that split's training complexes. For each node or edge feature, all training values were pooled across residues or edges, truncated to the training 0.5th and 99.5th percentiles, centred by the training mean, scaled by the training standard deviation, and finally clipped to the z-score range $[-8, 8]$. The same frozen transformation was then applied to the validation and test complexes.

3.5 Models

Both model families used the same residue-graph encoder architecture. The supervised baseline predicted affinity directly from the pooled graph representation, whereas the GraphVAE learned a masked latent representation and affinity was then predicted from the exported latent means with ridge regression.

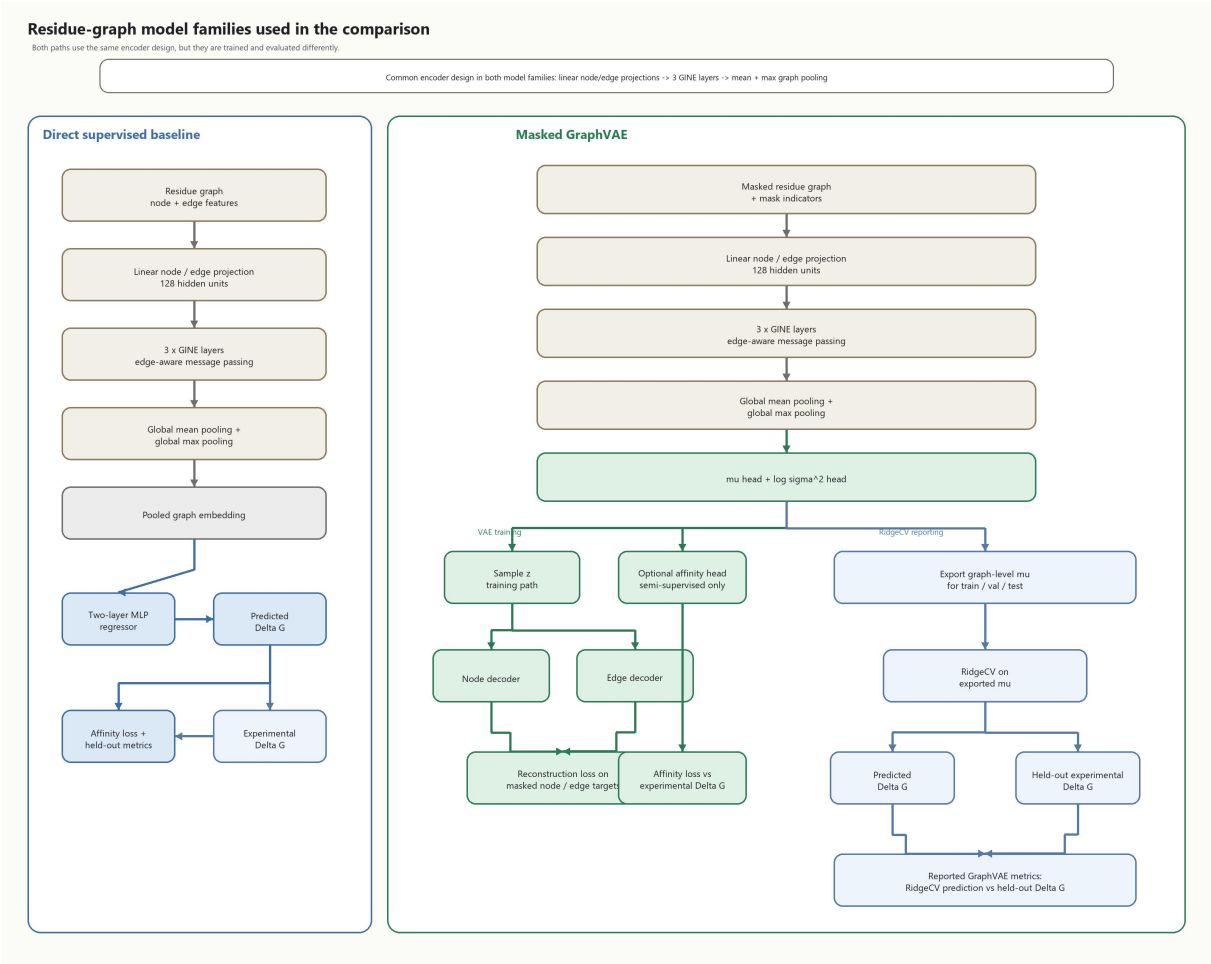

Figure 3.1: Model structure used in the comparison. The two model families use the same encoder architecture: linear node and edge projections, three GINE layers, and mean-plus-max graph pooling. The supervised baseline predicts ΔG directly from the pooled graph embedding. The GraphVAE maps the pooled graph embedding to μ and $\log \sigma^2$, uses sampled latent vectors for masked reconstruction, uses the latent mean for the optional affinity head during training, and reports affinity from ridge regression on exported latent means.

Figure 3.1 summarises the shared encoder backbone and the two model paths used in the comparison.

3.5.1 Direct supervised baseline

The supervised baseline was a residue-graph regressor in PyTorch Geometric [32]. Its task is simple: compress each residue graph to one graph-level vector and predict ΔG directly from that vector. Node and edge inputs were projected linearly to 128 hidden units, passed through three graph isomorphism network layers with edge features (GINE), pooled with the concatenation of global mean pooling and global max pooling, and mapped to ΔG by a two-layer multilayer perceptron [33, 34]. Across all baseline variants, the architecture and training settings were fixed: three GINE layers, hidden dimension 128, batch size 16,

the AdamW optimiser, and the same early-stopping rule.

3.5.2 Masked GraphVAE

The GraphVAE used the same encoder architecture as the supervised baseline: linear node and edge projections to 128 hidden units, three GINE layers, and mean-plus-max graph pooling. Two linear heads mapped the pooled representation to the latent mean μ and log-variance $\log \sigma^2$. The latent dimensionality was one of $d_z \in \{8, 16, 32\}$. The masking task trains the model to fill in deliberately hidden feature values from the rest of the graph. The goal is to learn a compact graph representation for affinity prediction.

For every batch, target node entries and target edge entries were masked independently with probability 0.30. The encoder received the masked graph together with binary mask indicators. The node decoder received the graph latent, the unmasked non-target node inputs, the masked target node inputs, and the node-mask indicators. The edge decoder received the analogous edge quantities. The reconstruction task was defined feature by feature for selected node and edge channels, instead of asking the model to generate an entire graph from scratch [21, 22].

The reconstruction targets used in the 52-model comparison were:

- **Static-only target:** buried surface area, solvent-accessible surface area, residue depth, the three half-sphere exposure channels, same-chain and partner-chain neighbour counts within 8 Å, and edge distance.
- **Dynamic-only target:** all dynamic node features and all dynamic edge features.
- **Static-plus-dynamic target:** the union of the static and dynamic target sets.

Residue-identity indicators and the same-chain edge flag were input features only; they were excluded from the reconstruction targets.

For a batch of graphs, the GraphVAE objective was

$$\mathcal{L} = \mathcal{L}_{\text{node}} + \mathcal{L}_{\text{edge}} + \beta \mathcal{L}_{\text{KL}} + 0.01\, \mathcal{L}_{\text{corr}} + \lambda_{\text{aff}} \mathcal{L}_{\text{aff}}, \tag{3.2}$$

where β is the weight on the Kullback–Leibler regulariser, $\mathcal{L}_{\text{node}}$ and $\mathcal{L}_{\text{edge}}$ were masked mean-squared reconstruction losses, $\mathcal{L}_{\text{corr}}$ penalised the squared off-diagonal entries of the latent covariance matrix to discourage redundant latent coordinates, and $\mathcal{L}_{\text{aff}}$ was an MSE loss on normalised ΔG from a latent MLP head. Unsupervised runs used $\lambda_{\text{aff}} = 0$. Semi-supervised runs used $\lambda_{\text{aff}} = 1$. The Kullback–Leibler term was

$$\mathcal{L}_{\text{KL}} = \frac{1}{B} \sum_{b=1}^{B} \frac{1}{d_z} \sum_{j=1}^{d_z} \left[-\frac{1}{2} \left(1 + \log \sigma_{bj}^2 - \mu_{bj}^2 - \sigma_{bj}^2 \right) \right], \tag{3.3}$$

where B is the batch size and d_z is the latent dimensionality.

All GraphVAE models were trained with AdamW, learning rate 10^{-4}, weight decay 10^{-5}, batch size 16, maximum 120 epochs, and early-stopping patience 20. The KL weight increased linearly from 0 to 1 over the first 30% of training epochs. Checkpoint selection used validation reconstruction loss for unsupervised runs and validation reconstruction loss plus weighted validation affinity loss for semi-supervised runs.

3.5.3 Latent-space ridge regression

GraphVAE results are reported from linear ridge regression on the latent means. The neural affinity head was used during semi-supervised training, but the headline evaluation asks a narrower question: how much affinity signal is already present in the learned representation? Using the same simple downstream model for every GraphVAE checkpoint keeps the comparison tied to latent quality instead of the flexibility of a second neural regressor. After training, each retained checkpoint exported graph-level latent means for the train, validation, and test complexes. A ridge regressor, that is, a linear model with ℓ_2 regularisation, was then fit on the training latents with `RidgeCV` from scikit-learn [35] over

$$\alpha \in \{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}, 3 \times 10^{-2}, 10^{-1}, 3 \times 10^{-1}, 1, 3, 10, 30, 100\}.$$

The headline GraphVAE metrics in Chapter 4 are the held-out metrics of this ridge regression.

3.6 Repeated outer evaluation

The 52-model comparison was evaluated on the matched 1437-complex set with 5 outer folds and 2 repeats. Repeated outer resampling was used because a single split can give a misleading picture of comparative performance when many closely related models are tested [36]. Resampling operated at the level of complexes, so all trajectory frames from a complex stayed in the same split. In each outer fit, one fold was held out as test and the remaining four folds were split into training and validation sets with validation fraction 0.1875 of the training-plus-validation pool, giving an overall train/validation/test proportion of approximately 65/15/20. At each outer fit, feature scaling, model training, checkpoint selection, and latent ridge fitting were all carried out within that split.

The comparison contained 52 models. Forty-eight were GraphVAE models spanning two graph views (full and interface-restricted), two feature modes (S and SD), two supervision modes (unsupervised and semi-supervised), and latent dimensions 8, 16, and 32. The SD runs used three reconstruction targets; the S runs used only the static-only target. Four further models were direct supervised baselines for the Cartesian product

of graph view and feature mode. Throughout the results chapter, S denotes static-only inputs, SD denotes static-plus-dynamic inputs, and d_z or z denotes latent dimension.

Held-out performance was measured with RMSE, Pearson correlation, and R^2. Every metric reported in Chapter 4 is the arithmetic mean over the 10 outer fits for the corresponding model, and the committed result snapshot also preserves the corresponding standard deviations.

Chapter 4

Results

The final evaluation used the 1437-complex analysis subset for which paired full and interface-restricted graph views were available. The comparison contained 52 models: 48 GraphVAE variants and 4 direct supervised baselines. Every reported metric is the mean over 10 outer held-out fits from 5 folds and 2 repeats. Here S denotes static-only inputs, SD static-plus-dynamic inputs, *semi* semi-supervised GraphVAE training, *unsup* unsupervised GraphVAE training, and z latent dimension. The complete 52-model table is given in Appendix B.

4.1 Headline models

The full static supervised baseline was the best overall model in the 52-model comparison. Figure 4.1 shows both the top of the model ranking and the main matched factor comparisons. Table 4.1 records the headline numerical values. The best GraphVAE was the full static semi-supervised model with latent dimension 32. The best interface model was the static supervised baseline.

The best supervised baseline outperformed the best GraphVAE by 0.0913 RMSE units and 0.0415 Pearson-correlation units on the mean metrics. Across the 10 outer fits, it had lower test RMSE in 8 of 10 splits than the best GraphVAE.

4.2 Factor comparisons

Graph view had the strongest effect: the full graph outperformed the interface-restricted graph in all 24 matched GraphVAE comparisons. Under the static-only reconstruction target, the static representation outperformed the static-plus-dynamic representation in 11 of 12 matched GraphVAE comparisons. Semi-supervision helped consistently for the full-graph runs and the interface static runs. For the interface dynamic runs, its effect was mixed: mean correlation improved, but mean RMSE was slightly worse. The family-level

Model	Test RMSE	Test r	Test R^2	Test−Train RMSE
Full baseline, S	2.3379 ± 0.1067	0.5064 ± 0.0567	0.2146 ± 0.0891	0.8021
Full baseline, SD	2.4073 ± 0.1290	0.4802 ± 0.0494	0.1685 ± 0.0876	0.8414
Full GraphVAE, S, semi, $z = 32$	2.4292 ± 0.0809	0.4649 ± 0.0363	0.1537 ± 0.0674	0.7451
Full GraphVAE, SD, semi, static-only, $z = 8$	2.4364 ± 0.1065	0.4749 ± 0.0529	0.1478 ± 0.0860	1.1303
Interface baseline, S	2.5782 ± 0.1120	0.3451 ± 0.0572	0.0470 ± 0.0820	0.8090
Interface GraphVAE, S, semi, $z = 8$	2.6041 ± 0.1019	0.3402 ± 0.0611	0.0282 ± 0.0718	0.8018

Table 4.1: Best-performing models in the 52-model comparison. Reported RMSE, Pearson r, and R^2 values are mean ± standard deviation over the 10 outer fits. GraphVAE metrics are those obtained after ridge regression on latent means.

Comparison	Pairs	Mean RMSE gain for first option	Mean Pearson r gain for first option	Pair wins for first option
Full over interface	24	0.1005	0.0993	24 / 24
S over SD (shared static)	12	0.0254	0.0215	11 / 12
Semi over unsup, full S	3	0.1640	0.2497	3 / 3
Semi over unsup, full SD	9	0.1604	0.3339	9 / 9
Semi over unsup, interface S	3	0.0290	0.2436	3 / 3
Semi over unsup, interface SD	9	-0.0081	0.2766	3 / 9

Table 4.2: Matched GraphVAE comparisons. A positive RMSE gain means the first-named option has lower mean test RMSE.

results later in the chapter help interpret the dynamic-feature result: in several semi-supervised families, SD models fit training data more strongly while still giving weaker held-out performance. Table 4.2 gives the numerical summary, while Figure 4.1 gives the same comparisons visually.

Within the full semi-supervised SD family, the reconstruction target also changed performance. Averaged over latent dimensions 8, 16, and 32, the mean test RMSE was 2.4434 for the static-only target, 2.4795 for the static-plus-dynamic target, and 2.5150 for the dynamic-only target. The corresponding mean test Pearson correlations were 0.4682, 0.4335, and 0.4221.

4.3 Stability under repeated evaluation

The fourth research question concerned sensitivity to the split. The main qualitative conclusions stayed stable across repeated evaluation, although the margin between the two best full-graph models was modest. The full static baseline achieved mean test RMSE 2.3379 ± 0.1067, while the best GraphVAE achieved 2.4292 ± 0.0809. Across the 10 outer fits, the full static baseline had lower test RMSE than the best GraphVAE in 8 of 10 splits and lower test RMSE than the interface static baseline in 9 of 10 splits. The full-over-interface effect was even clearer at the design level, because the full graph won all 24 matched GraphVAE comparisons.

The same ranking and the same directional factor effects recurred across multiple train/test partitions, even though the top two full-graph models stay close on a few splits.

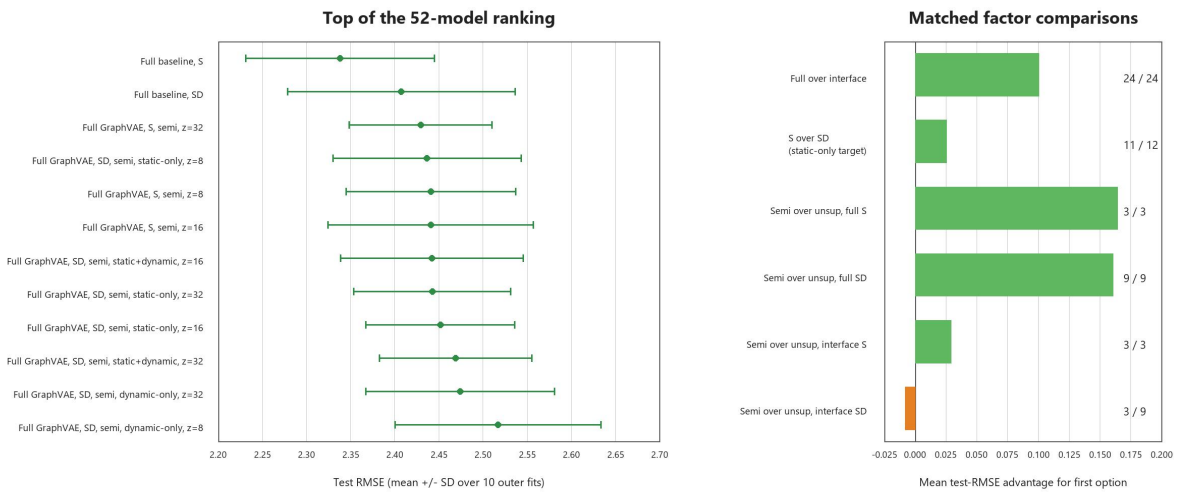

Figure 4.1: Main empirical picture of the 52-model comparison. Left: top of the model ranking by mean test RMSE with one-standard-deviation error bars over the 10 outer fits. Right: matched GraphVAE factor comparisons, reported as the mean test-RMSE advantage for the first-named option.

4.4 Family-level train, validation, and test behaviour

Table 4.3 summarises the main model families. The unsupervised VAE families had small train–test gaps but weak correlations on all splits, which is more consistent with underfitting than with strong regularised prediction. The semi-supervised families fit the training set more strongly. Within those semi-supervised families, adding dynamic summaries often increased train correlation and reduced train RMSE, yet those gains stayed on the training set and faded on validation and test. The strongest held-out result remained the full static supervised baseline.

The clearest example is the full semi-supervised GraphVAE family: moving from S to SD raised mean train correlation from 0.7494 to 0.8218 and lowered mean train RMSE from 1.7385 to 1.4744, yet mean test correlation fell from 0.4551 to 0.4413 and mean test RMSE worsened from 2.4369 to 2.4793. The same pattern appears in the interface semi-supervised families. The dynamic summaries gave the models more variation to fit, but that extra fit stayed on the training set instead of improving held-out prediction.

4.5 Performance by interaction class

The matched 1437-complex analysis set contained 966 complexes in the broad *other* class, 366 antibody–antigen complexes, and 105 TCR-pMHC complexes. For subgroup analysis, the outer-test predictions were pooled over all 10 fits, so each complex contributes two held-out predictions, one from each repeat.

Table 4.4 shows the subgroup behaviour of the main models, and Figure 4.2 adds

Family	Train r	Val r	Test r	g_r	Train RMSE	Test RMSE
Full baseline S	0.8225	0.5337	0.5064	0.3161	1.5358	2.3379
Full baseline SD	0.8077	0.5191	0.4802	0.3275	1.5659	2.4073
Full VAE unsup S	0.2486	0.1897	0.2055	0.0431	2.5734	2.6009
Full VAE unsup SD	0.1809	0.0967	0.1074	0.0736	2.6062	2.6398
Full VAE semi S	0.7494	0.4666	0.4551	0.2942	1.7385	2.4369
Full VAE semi SD	0.8218	0.4580	0.4413	0.3805	1.4744	2.4793
Interface VAE unsup S	0.1495	0.0794	0.0925	0.0570	2.6289	2.6402
Interface VAE unsup SD	0.1261	0.0382	0.0424	0.0838	2.6324	2.6538
Interface VAE semi S	0.7083	0.3356	0.3361	0.3722	1.8467	2.6112
Interface VAE semi SD	0.7685	0.3231	0.3189	0.4496	1.6479	2.6620

Table 4.3: Family-level mean train, validation, and test behaviour. Here $g_r = r_{\text{train}} - r_{\text{test}}$. Each row is averaged over all models in that family.

Subgroup	n	Full baseline RMSE	Full baseline r	Full GraphVAE RMSE	Full GraphVAE r	Interface baseline RMSE	Interface baseline r
Antibody–antigen	366	2.048	0.292	2.134	0.193	2.356	0.089
Other	966	2.501	0.430	2.584	0.394	2.675	0.312
TCR-pMHC	105	1.663	0.466	1.879	0.406	2.440	0.333

Table 4.4: Pooled outer-test subgroup metrics for the best overall model (full static baseline), the best GraphVAE (full static semi-supervised GraphVAE, latent dimension 32), and the best interface model (interface static baseline).

the outer-fit variability. The full static baseline remained the strongest of these models in all three interaction classes. The full static GraphVAE remained second, and the interface baseline remained weaker in every subgroup. The weakest correlations were in the antibody–antigen subgroup, while the TCR-pMHC subgroup had the smallest sample size and therefore the widest uncertainty.

4.6 Summary

Four results dominate the comparison. First, the full graph was better than the interface-restricted representation. Second, the dynamic summaries offered no held-out gain over the static representation, even though they sometimes improved in-sample fit. Third, the strongest model was the direct supervised baseline on the full static graph. Fourth, the same model ordering remained in the antibody–antigen, TCR-pMHC, and other interaction classes, and the repeated-resampling analysis showed that these conclusions hold across multiple held-out splits.

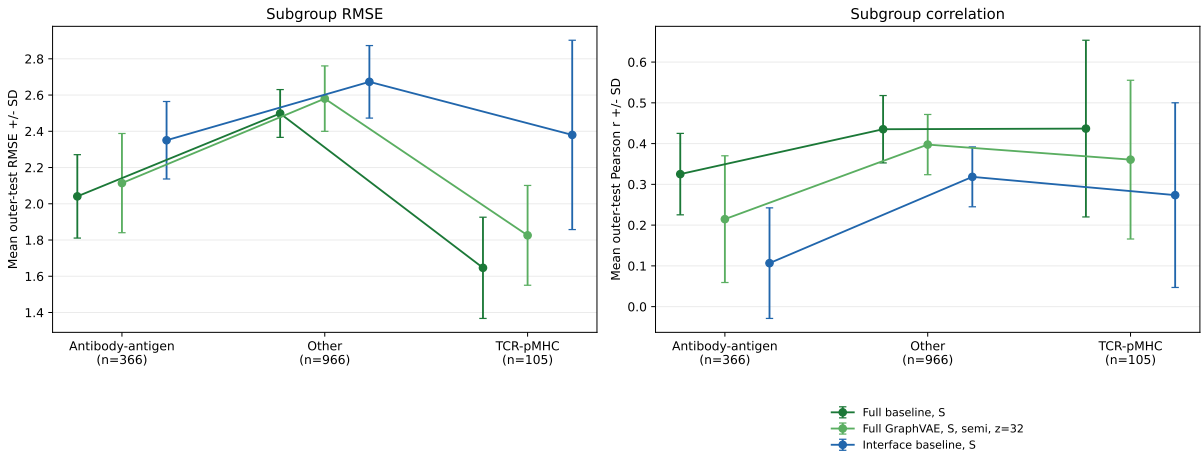

Figure 4.2: Subgroup performance of the three headline models. Points show the mean outer-test metric and error bars show one standard deviation across the 10 outer fits. The model ordering is stable across subgroups, but uncertainty is visibly larger for the 105-complex TCR-pMHC subset.

Chapter 5

Discussion and Conclusions

5.1 Main findings

First, the full residue graph outperformed the interface-restricted representation. The full view won all 24 matched GraphVAE comparisons and both supervised baseline comparisons. Whole-complex context appears to help the model recognise which partners are present, how the two chains are arranged in space, and what broader structural setting surrounds the binding site. A fixed 128-residue interface window keeps only part of that picture. On this benchmark, useful affinity signal extended beyond the interface alone. Once the model could process the whole residue graph, the extra context was more useful than a tighter focus on the contact region. This contrasts with interface-focused predictors such as PRODIGY and the PPB-Affinity benchmark model and is closer in spirit to whole-complex or hybrid approaches such as Moal et al., PPI-Affinity, and PPI-Graphomer [1, 7–9, 17]. Under the same encoder family and repeated held-out evaluation, the whole-complex view was the stronger choice.

Second, the static feature set remained stronger than the MD-derived summaries. The comparison between S and SD was intentionally controlled: both modes started from the same post-prep graph, so the only extra information in SD came from the trajectory summaries. Within that shared setup, the family-level results show the same pattern in both the full and interface semi-supervised families. Adding dynamic summaries improved train fit but weakened validation and test performance. The added MD summaries therefore gave the models more variation to fit without improving held-out prediction.

Third, the masked GraphVAE remained competitive, and semi-supervised training improved most of the VAE families over the unsupervised variants. The exception was the interface dynamic family, where the effect was mixed. The full static baseline still produced the strongest held-out predictor. A likely explanation is that direct supervised training can spend all of its capacity on affinity, whereas the VAE also spends capacity reconstructing masked graph features. That trade-off still produced useful latent spaces,

but direct supervised training was more effective for affinity prediction in this comparison [20, 21, 24].

Fourth, the main conclusions were stable under repeated held-out evaluation. The same design choices stayed favourable across 10 outer fits, and the family-level summaries support the same ranking as the leading models. The full static baseline beat the best GraphVAE in 8 of 10 outer fits, so the top of the ranking is consistent even though the two best full-graph models remain fairly close.

Different interaction classes also remained visibly different in the final comparison. On pooled outer-test predictions, the best overall model, the full static baseline, gave Pearson $r = 0.292$ on antibody–antigen, $r = 0.430$ on the broad other class, and $r = 0.466$ on TCR-pMHC. The full static baseline remained the strongest of the main models in all three interaction classes. This matches PPB-Affinity's subgroup behaviour and later antibody-specific work arguing that antibody–antigen complexes benefit from specialised treatment [1, 14]. These subgroup gaps are part of the scientific picture in their own right. A model can look competitive on the pooled benchmark and still struggle on an interaction class that matters in practice. The antibody–antigen correlations here are clearly weaker than the broad other class, while the TCR-pMHC subgroup is small enough that its uncertainty remains large.

5.2 Limitations

Trajectory length was the main limitation. Each complex contributed 1.2 ns of saved MD, which is enough to capture local fluctuation and contact persistence but is short relative to many rearrangements that shape protein binding [5, 6, 13]. Slow collective motions often extend to microseconds and slower [6], so these trajectories were unlikely to sample much of the slower motion that distinguishes one binding ensemble from another. That makes limited added signal the most likely explanation for the weak dynamic-feature results.

The dynamic features also carry noise from how the MD was run and summarised. In an ideal study, preparation would be refined case by case and the trajectories would be kept in a richer form. That would have reduced the number of complexes that could be carried through the comparison. To keep the study large enough for machine learning, preparation was standardised across complexes, missing residues were not rebuilt, protonation was standardised, and each short trajectory was compressed to residue- and edge-level summary statistics on a fixed graph. Those choices made a 1437-complex comparison feasible, but they also combine thermal fluctuation, finite sampling, and summary-statistic noise. Using the same post-prep graph for both S and SD reduced noise in the comparison itself, because both modes started from the same structure source. That same choice may also leave the absolute static-only numbers below what graphs built directly from raw experimental coordinates could achieve.

The final controlled comparison also used a 1437-complex matched subset drawn from the 2818-complex curated table. That retained set is enriched for antibody–antigen complexes and slightly stronger binders, so the reported numbers should be read as results on this matched subset rather than as a direct replacement for the full PPB-Affinity benchmark. The repeated resampling was done by complex identifier only, leaving homology-aware and structure-aware separation unconstrained, so related protein families could still appear across train and test. Earlier protein–protein studies removed homologous complexes for this reason, and similar leakage problems have been documented in protein–ligand affinity prediction [7, 19]. Transfer to unbound or predicted inputs remains open, since the present study uses bound complex structures in which both proteins are already observed together [2].

5.3 Future work

Future work should give MD a better chance to help. That means longer or better-targeted trajectories, especially if the goal is to capture slower binding-related motion as well as local fluctuation. It also means cleaner structure preparation where the compute budget allows it. Recent protein–ligand work suggests that MD becomes more useful when the model keeps richer temporal or ensemble information instead of reducing the trajectory to a few averages [10, 11].

Future work should also test richer structure and evaluation settings. The current study used one shared post-prep graph for both S and SD to isolate the incremental effect of the trajectory summaries. Comparing that controlled design with graphs built directly from raw experimental coordinates would show how much the shared post-prep source affects absolute static performance. It would also be useful to evaluate stricter family-aware splits and to test the same representation choices on unbound or predicted structures, where practical affinity prediction is often harder.

5.4 Conclusion

Overall, this study shows that a full static residue graph is a strong baseline for protein–protein affinity prediction on this benchmark. In the 52-model comparison, that baseline remained stronger than the short-MD summary variants. The main lesson is that, in this implementation, the trajectories were too short to add much new affinity signal, and the coarse MD summaries then made that weak signal difficult to use reliably.

Appendix A

Archived Approaches and Project Evolution

This appendix records the main alternative approaches explored during the project. These archived branches show other directions that were considered before the final thesis focused on residue graphs with short-MD summary features. The materials are preserved under `ARCHIVE` in the repository and can also be browsed at `https://github.com/RohanDevereux/Bridging/tree/main/ARCHIVE`.

A.1 Correction learning around fast affinity baselines

An early project idea was to predict a correction

$$\delta = \Delta G_{\text{exp}} - \Delta G_{\text{base}}$$

instead of predicting ΔG directly. Two approaches were explored in this form: a contact-based baseline using PRODIGY [17] and an endpoint free-energy baseline using MM/GBSA [16].

A.1.1 PRODIGY

One approach used PRODIGY estimates as a fast structural baseline and then learned a correction to experimental affinity [17]. The corresponding materials are preserved under `ARCHIVE/PRODIGY`. In the saved runs, raw PRODIGY predictions were weak on the archived TCR-heavy subsets, and direct prediction from learned structural representations was stronger than learning a correction around PRODIGY. The archived subgroup summaries also already hinted that the interaction classes behave differently.

The saved subset-specific baseline outputs showed the same pattern. On an archived TCR-only dataset, PRODIGY was essentially uncorrelated with experiment. On a small

Archived subset	n	RMSE	MAE	Pearson r	Spearman ρ
TCR-pMHC	65	9.19	7.12	−0.051	−0.044
Antibody–antigen	70	2.33	1.80	0.243	0.236
Diverse100 mixed set	100	6.75	4.66	−0.175	−0.086

Table A.1: Saved subset-specific PRODIGY results from archived curated datasets. The TCR-pMHC subset was especially weak, while the small antibody–antigen subset was somewhat better.

Archived regression variant	PRODIGY n	PRODIGY RMSE	PRODIGY r	Direct RMSE	Direct r	Corrected RMSE
Early contact-map latent regressor	13	10.50	−0.268	6.92	−0.018	8.64
Set-based edge encoder, 24 epochs	13	10.50	−0.268	2.80	−0.500	7.75

Table A.2: Representative archived PRODIGY-based regression runs on the TCR subset. The direct model is the learned latent-space regressor, while the corrected model tries to learn an adjustment to PRODIGY.

antibody–antigen archived dataset, the same baseline was better, though still modest. A mixed 100-complex subset was again weak.

The next table switches from raw baseline evaluation to the correction-learning setup. It keeps the same PRODIGY baseline column and compares two learned routes on overlapping archived TCR test slices: direct prediction from the learned representation, and prediction of a correction around PRODIGY.

A.1.2 MM/GBSA

A second correction-learning approach used MM/GBSA endpoint estimates [16]. The corresponding materials are preserved under `ARCHIVE/MMGBSA`. Across the partial result files there were 163 rows with status `ok`. On those comparable rows, the MM/GBSA estimates had Pearson correlation −0.127, Spearman correlation −0.096, RMSE 2458.69, and MAE 1398.31. The scale of the predictions was far from the experimental target range: the median successful estimate was 466.38 kcal mol^{-1}, the 95th percentile was 5590.71 kcal mol^{-1}, and the maximum exceeded 13 000 kcal mol^{-1}, whereas the corresponding experimental free energies lay roughly between −21 and −4 kcal mol^{-1}.

A.2 The Cα contact-map approach

An earlier model family represented complexes as fixed two-dimensional tensors built from Cα–Cα contact and distance channels, followed by a convolutional variational autoencoder and regression model. The corresponding materials are preserved under `ARCHIVE/CA_contact_map`. This image-like representation was appealing because related ideas have worked in protein–ligand learning [18], but in the archived runs it was too unstable for the present task. Some variants reduced RMSE relative to raw PRODIGY on small subsets,

Quantity	Value
Successful rows	163
Comparable successful rows	163
Pearson correlation	−0.127
Spearman correlation	−0.096
RMSE	2458.69
MAE	1398.31
Median predicted MM/GBSA ΔG	466.38
95th percentile predicted MM/GBSA ΔG	5590.71
Maximum predicted MM/GBSA ΔG	13628.15

Table A.3: MM/GBSA partial-result summary from the archived approach. The estimates were far outside the scale of experimental protein–protein binding free energies, so this route was dropped as a quantitative baseline.

Run	Test n	Direct-latent RMSE	Direct-latent Pearson r
`setedge_cpu`	8	3.03	−0.065
`setedge_e24`	12	2.80	−0.500
`setedge_e24_ols`	12	2.76	−0.527
`setedge_e24_seed1`	12	6.62	−0.124
`setedge_e24_seed4`	12	3.17	0.462
`setedge_semisup`	12	5.48	0.072
`allmd_setedge_e24`	12	4.23	0.087

Table A.4: Representative Cα contact-map regression runs on the archived TCR subset. Some runs reduced RMSE relative to raw PRODIGY, but the results were unstable and often had weak or negative test correlation.

yet the same runs often had weak or negative test correlations and varied sharply across seeds. The reported values below are test-set outputs from the direct-latent regression path built on that contact-map representation.

Table A.4 reports representative direct-latent regressors from that family. The run names differ by training schedule, regression variant, or random seed, but they share the same basic contact-map representation. The spread across those runs explains why the final comparison moved on from this path.

Compared with the final residue-graph formulation, the contact-map representation threw away residue identity detail, side-chain context, and explicit graph structure while still depending on a fixed interface crop and a fixed tensor ordering. The residue-graph formulation retained more structural information and connected more naturally to the MD-derived features used in the main text, so the project later shifted away from the contact-map path.

A.3 PPB-Affinity interface-attention augmentation

Another line of work aimed to use MD more directly, with baseline, frame-level, and trajectory-level models together with an ensemble stage. The corresponding materials

Component	Role in the archived augmentation branch
Single-structure baseline	One structure per complex, passed through the PPB-Affinity-style interface-attention model as the non-augmented reference path.
Frame augmentation	Saved MD frames reused as additional supervised inputs, each carrying the parent complex's affinity label.
Trajectory model	Ordered frame embeddings passed through a BiGRU before regression, so temporal order was retained instead of collapsed to simple averages.
Ensemble stage	Intended to aggregate predictions across the baseline, frame-augmented, and trajectory-level outputs.

Table A.5: Core ideas in the archived PPB-Affinity augmentation branch.

are preserved under `ARCHIVE/PPB_Affinity_augmentation`. This branch reused a vendored PPB-Affinity-style baseline stack as its starting point, including the same kind of geometry-aware attention model over an interface patch, and then changed the data path around it. Here, augmentation meant treating saved MD frames from one complex as additional supervised inputs that all inherited the same affinity label. The branch then explored three linked routes on top of that PPB-style interface model: a single-structure baseline, frame-level models trained on those augmented examples, and trajectory-level models that passed ordered frame embeddings through a bidirectional gated recurrent unit (BiGRU) before final regression [37, 38]. Recent protein–ligand work had started to learn from MD frame sequences rather than from simple averages [11], so this branch tested whether the same idea could transfer to protein–protein affinity prediction.

This branch remained exploratory because the project later shifted toward the residue-graph comparison used in the main text. That change in direction says little about the augmentation idea itself. It is still relevant as a follow-on direction because it preserves temporal order that the final summary-feature approach discards.

Appendix B

Complete Model Matrix

This appendix gives the complete 52-model evaluation matrix, sorted by mean test RMSE. A machine-readable copy is stored in the results directory as `final_resample_matrix_summary.csv`. Here *Sup.* denotes supervision status, *Gap* denotes the mean test-minus-train RMSE gap, and the machine-readable label *shared static* denotes the static-only reconstruction target used in the main text.

Table B.1: Complete 52-model evaluation matrix, sorted by mean test RMSE. Metrics are means over 10 outer held-out fits.

View	Model	Mode	Sup.	Target	z	RMSE	r	R^2	Gap
Full	Baseline	S	sup	–	–	2.3379	0.5064	0.2146	0.8021
Full	Baseline	SD	sup	–	–	2.4073	0.4802	0.1685	0.8414
Full	GraphVAE	S	semi	shared static	32	2.4292	0.4649	0.1537	0.7451
Full	GraphVAE	SD	semi	shared static	8	2.4364	0.4749	0.1478	1.1303
Full	GraphVAE	S	semi	shared static	8	2.4407	0.4507	0.1451	0.6921
Full	GraphVAE	S	semi	shared static	16	2.4407	0.4499	0.1438	0.6580
Full	GraphVAE	SD	semi	static+dynamic	16	2.4419	0.4503	0.1437	0.8952
Full	GraphVAE	SD	semi	shared static	32	2.4423	0.4669	0.1443	1.2169
Full	GraphVAE	SD	semi	shared static	16	2.4516	0.4629	0.1354	1.1312
Full	GraphVAE	SD	semi	static+dynamic	32	2.4687	0.4411	0.1233	0.8527
Full	GraphVAE	SD	semi	dynamic only	32	2.4740	0.4508	0.1231	1.0153
Full	GraphVAE	SD	semi	dynamic only	8	2.5168	0.4202	0.0920	0.9580
Full	GraphVAE	SD	semi	static+dynamic	8	2.5277	0.4092	0.0818	0.7819
Full	GraphVAE	SD	semi	dynamic only	16	2.5543	0.3954	0.0629	1.0628
Interface	Baseline	S	sup	–	–	2.5782	0.3451	0.0470	0.8090
Interface	Baseline	SD	sup	–	–	2.5829	0.3347	0.0463	0.6954
Full	GraphVAE	S	unsup	shared static	32	2.5906	0.2271	0.0401	0.0617
Interface	GraphVAE	S	semi	shared static	8	2.6041	0.3402	0.0282	0.8018
Full	GraphVAE	S	unsup	shared static	16	2.6052	0.1910	0.0303	0.0194
Full	GraphVAE	S	unsup	shared static	8	2.6069	0.1984	0.0286	0.0016
Interface	GraphVAE	S	semi	shared static	32	2.6113	0.3364	0.0207	0.7354
Interface	GraphVAE	S	semi	shared static	16	2.6180	0.3316	0.0166	0.7561
Interface	GraphVAE	SD	semi	static+dynamic	16	2.6238	0.3252	0.0146	0.7597
Full	GraphVAE	SD	unsup	shared static	16	2.6313	0.1259	0.0102	0.0265
Full	GraphVAE	SD	unsup	shared static	8	2.6322	0.1249	0.0103	0.0124
Interface	GraphVAE	S	unsup	shared static	32	2.6324	0.1211	0.0102	0.0118
Full	GraphVAE	SD	unsup	static+dynamic	32	2.6328	0.1255	0.0093	0.0610
Full	GraphVAE	SD	unsup	dynamic only	32	2.6353	0.1100	0.0074	0.0364
Full	GraphVAE	SD	unsup	shared static	32	2.6365	0.1813	0.0039	0.1021
Interface	GraphVAE	SD	semi	dynamic only	16	2.6366	0.3298	0.0032	0.9460
Interface	GraphVAE	S	unsup	shared static	8	2.6427	0.0790	0.0025	0.0062
Full	GraphVAE	SD	unsup	dynamic only	16	2.6441	0.0981	0.0013	0.0292
Interface	GraphVAE	S	unsup	shared static	16	2.6454	0.0774	0.0003	0.0157
Full	GraphVAE	SD	unsup	static+dynamic	8	2.6454	0.0719	0.0006	0.0048
Interface	GraphVAE	SD	semi	shared static	16	2.6455	0.3362	-0.0015	1.1836

View	Model	Mode	Sup.	Target	z	RMSE	r	R^2	Gap
Full	GraphVAE	SD	unsup	dynamic only	8	2.6461	0.0706	0.0000	0.0013
Interface	GraphVAE	SD	unsup	shared static	8	2.6473	0.0618	-0.0010	0.0129
Interface	GraphVAE	SD	unsup	static+dynamic	8	2.6481	0.0552	-0.0016	0.0056
Interface	GraphVAE	SD	unsup	shared static	32	2.6530	0.0896	-0.0051	0.0451
Interface	GraphVAE	SD	unsup	static+dynamic	32	2.6534	0.0268	-0.0056	0.0222
Full	GraphVAE	SD	unsup	static+dynamic	16	2.6542	0.0582	-0.0060	0.0286
Interface	GraphVAE	SD	unsup	dynamic only	8	2.6552	0.0293	-0.0071	0.0097
Interface	GraphVAE	SD	unsup	dynamic only	16	2.6554	0.0235	-0.0070	0.0151
Interface	GraphVAE	SD	semi	dynamic only	8	2.6557	0.3279	-0.0116	1.0207
Interface	GraphVAE	SD	unsup	static+dynamic	16	2.6557	0.0209	-0.0073	0.0127
Interface	GraphVAE	SD	unsup	shared static	16	2.6575	0.0395	-0.0088	0.0353
Interface	GraphVAE	SD	unsup	dynamic only	32	2.6589	0.0347	-0.0099	0.0347
Interface	GraphVAE	SD	semi	shared static	8	2.6684	0.3270	-0.0217	1.1249
Interface	GraphVAE	SD	semi	shared static	32	2.6705	0.3185	-0.0230	1.2466
Interface	GraphVAE	SD	semi	static+dynamic	8	2.6743	0.3039	-0.0243	0.8622
Interface	GraphVAE	SD	semi	dynamic only	32	2.6855	0.3076	-0.0322	1.0470
Interface	GraphVAE	SD	semi	static+dynamic	32	2.6974	0.2941	-0.0411	0.9357

Appendix C

Repository Layout

The source code for the study is hosted at `https://github.com/RohanDevereux/Bridging`

The main repository layout is:

- Molecular-dynamics preparation and simulation:
  `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/MD`
- Residue-graph preparation:
  `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/graphvae/prep`
- GraphVAE, supervised baseline, and latent-ridge evaluation:
  `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/graphvae/ml`
- Command-line runners for the final evaluation:
  `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/graphvae/runners`
- Shared affinity-conversion and dataset utilities:
  `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/utils`
- Curated datasets used in the main analysis:
  `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/processedData`
- Final 52-configuration matrix summary:
  `https://github.com/RohanDevereux/Bridging/blob/main/RESULTS/final_resample_matrix_summary.csv`
- Saved subgroup analysis from the raw outer-fold prediction files:
  `https://github.com/RohanDevereux/Bridging/tree/main/RESULTS/final_resample_subgroups`

Appendix C. Repository Layout

- Subgroup-export script:
  `https://github.com/RohanDevereux/Bridging/blob/main/src/bridging/graphvae/tools/export_final_resample_subgroups.py`
- Archived branches discussed in Appendix A:
  `https://github.com/RohanDevereux/Bridging/tree/main/ARCHIVE`

Bibliography

[1] Huaqing Liu, Peiyi Chen, Xiaochen Zhai, Ku-Geng Huo, Shuxian Zhou, Lanqing Han, and Guoxin Fan. PPB-Affinity: Protein-protein binding affinity dataset for AI-based protein drug discovery. *Scientific Data*, 11(1):1316, 2024. doi: 10.1038/s41597-024-03997-4.

[2] Zhongliang Guo and Rui Yamaguchi. Machine learning methods for protein-protein binding affinity prediction in protein design. *Frontiers in Bioinformatics*, 2:1065703, 2022. doi: 10.3389/fbinf.2022.1065703.

[3] Ashley E. Modell, Sarah L. Blosser, and Paramjit S. Arora. Systematic targeting of protein-protein interactions. *Trends in Pharmacological Sciences*, 37(8):702–713, 2016. doi: 10.1016/j.tips.2016.05.008.

[4] Zoe Cournia, Bryce Allen, and Woody Sherman. Relative binding free energy calculations in drug discovery: Recent advances and practical considerations. *Journal of Chemical Information and Modeling*, 57(12):2911–2937, 2017. doi: 10.1021/acs.jcim.7b00564.

[5] Till Siebenmorgen and Martin Zacharias. Computational prediction of protein–protein binding affinities. *Wiley Interdisciplinary Reviews: Computational Molecular Science*, 10(3):e1448, 2020. doi: 10.1002/wcms.1448.

[6] Katherine Henzler-Wildman and Dorothee Kern. Dynamic personalities of proteins. *Nature*, 450:964–972, 2007. doi: 10.1038/nature06522.

[7] Iain H. Moal, Rudi Agius, and Paul A. Bates. Protein-protein binding affinity prediction on a diverse set of structures. *Bioinformatics*, 27(21):3002–3009, 2011. doi: 10.1093/bioinformatics/btr513.

[8] Sandra Romero-Molina, Yasser B. Ruiz-Blanco, Joel Mieres-Perez, Mirja Harms, Jan Münch, Michael Ehrmann, and Elsa Sanchez-Garcia. PPI-Affinity: A web tool for the prediction and optimization of protein-peptide and protein-protein binding affinity. *Journal of Proteome Research*, 21(8):1829–1841, 2022. doi: 10.1021/acs.jproteome.2c00020.

[9] Jun Xie, Youli Zhang, Ziyang Wang, Xiaocheng Jin, Xiaoli Lu, Shengxiang Ge, and Xiaoping Min. PPI-Graphomer: Enhanced protein-protein affinity prediction using pretrained and graph transformer models. *BMC Bioinformatics*, 26(1):116, 2025. doi: 10.1186/s12859-025-06123-2.

[10] Yaosen Min, Ye Wei, Peizhuo Wang, Xiaoting Wang, Han Li, Nian Wu, Stefan Bauer, Shuxin Zheng, Yu Shi, Yingheng Wang, Ji Wu, Dan Zhao, and Jianyang Zeng. From static to dynamic structures: Improving binding affinity prediction with graph-based deep learning. *Advanced Science*, 11(40):2405404, 2024. doi: 10.1002/advs.202405404.

[11] Pierre-Yves Libouban, Camille Parisel, Maxime Song, Samia Aci-Sèche, Jose C. Gómez-Tamayo, Gary Tresadern, and Pascal Bonnet. Spatio-temporal learning from molecular dynamics simulations for protein–ligand binding affinity prediction. *Bioinformatics*, 41(8):btaf429, 2025. doi: 10.1093/bioinformatics/btaf429.

[12] Renxiao Wang, Xiaolu Fang, Yipin Lu, and Shaomeng Wang. The PDBbind database: Collection of binding affinities for protein–ligand complexes with known three-dimensional structures. *Journal of Medicinal Chemistry*, 47(12):2977–2980, 2004. doi: 10.1021/jm030580l.

[13] Panagiotis L. Kastritis, Iain H. Moal, Howook Hwang, Zhiping Weng, Paul A. Bates, Alexandre M. J. J. Bonvin, and Joël Janin. A structure-based benchmark for protein-protein binding affinity. *Protein Science*, 20(3):482–491, 2011. doi: 10.1002/pro.580.

[14] Yong Xiao Yang, Pan Wang, and Bao Ting Zhu. Binding affinity prediction for antibody-protein antigen complexes: A machine learning analysis based on interface and surface areas. *Journal of Molecular Graphics and Modelling*, 118:108364, 2023. doi: 10.1016/j.jmgm.2022.108364.

[15] Glenn M. Torrie and John P. Valleau. Nonphysical sampling distributions in Monte Carlo free-energy estimation: Umbrella sampling. *Journal of Computational Physics*, 23(2):187–199, 1977. doi: 10.1016/0021-9991(77)90121-8.

[16] Iva Massova and Peter A. Kollman. Combined molecular mechanical and continuum solvent approach (MM-PBSA/GBSA) to predict ligand binding. *Perspectives in Drug Discovery and Design*, 18(1):113–135, 2000. doi: 10.1023/A:1008763014207.

[17] Anna Vangone and Alexandre M. J. J. Bonvin. Contacts-based prediction of binding affinity in protein-protein complexes. *eLife*, 4:e07454, 2015. doi: 10.7554/eLife.07454.

[18] Marta M. Stepniewska-Dziubinska, Piotr Zielenkiewicz, and Pawel Siedlecki. Development and evaluation of a deep learning model for protein–ligand binding affinity

prediction. *Bioinformatics*, 34(21):3666–3674, 2018. doi: 10.1093/bioinformatics/bty374.

[19] David Graber, Peter Stockinger, Fabian Meyer, Siddhartha Mishra, Claus Horn, and Rebecca Buller. Resolving data bias improves generalization in binding affinity prediction. *Nature Machine Intelligence*, 7(10):1713–1725, 2025. doi: 10.1038/s42256-025-01124-5.

[20] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations*, 2014. URL `https://arxiv.org/abs/1312.6114`. Originally released as arXiv:1312.6114.

[21] Thomas N. Kipf and Max Welling. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308*, 2016. URL `https://arxiv.org/abs/1611.07308`.

[22] Martin Simonovsky and Nikos Komodakis. GraphVAE: Towards generation of small graphs using variational autoencoders. In *Artificial Neural Networks and Machine Learning – ICANN 2018*, volume 11139 of *Lecture Notes in Computer Science*, pages 412–422, Cham, 2018. Springer. doi: 10.1007/978-3-030-01418-6_41.

[23] Rafael Gómez-Bombarelli, Jennifer N. Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamín Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre, Timothy D. Hirzel, Ryan P. Adams, and Alán Aspuru-Guzik. Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science*, 4(2):268–276, 2018. doi: 10.1021/acscentsci.7b00572.

[24] Hao Tian, Xi Jiang, Francesco Trozzi, Sian Xiao, Eric C. Larson, and Peng Tao. Explore protein conformational space with variational autoencoder. *Frontiers in Molecular Biosciences*, 8:781635, 2021. doi: 10.3389/fmolb.2021.781635.

[25] Justina Jankauskaitė, Brian Jiménez-García, Jūratė Dapkūnas, Juan Fernández-Recio, and Iain H. Moal. SKEMPI 2.0: An updated benchmark of changes in protein–protein binding energy, kinetics and thermodynamics upon mutation. *Bioinformatics*, 35(3):462–469, 2019. doi: 10.1093/bioinformatics/bty635.

[26] OpenMM Developers. PDBFixer, 2026. URL `https://github.com/openmm/pdbfixer`. Software repository; accessed 2026-03-25.

[27] Peter Eastman, Raimondas Galvelis, Raúl P. Peláez, Charlles R. A. Abreu, Stephen E. Farr, Emilio Gallicchio, Anton Gorenko, Michael M. Henry, Frank Hu, Jing Huang, Andreas Krämer, Julien Michel, Joshua A. Mitchell, Vijay S. Pande, João PGLM Rodrigues, Jaime Rodriguez-Guerra, Andrew C. Simmonett, Sukrit Singh, Jason Swails, Philip Turner, Yuanqing Wang, Ivy Zhang, John D. Chodera,

Gianni De Fabritiis, and Thomas E. Markland. OpenMM 8: Molecular dynamics simulation with machine learning potentials. *The Journal of Physical Chemistry B*, 128(1):109–116, 2024. doi: 10.1021/acs.jpcb.3c06662.

[28] James A. Maier, Carmenza Martinez, Koushik Kasavajhala, Lauren Wickstrom, Karl E. Hauser, and Carlos Simmerling. ff14SB: Improving the accuracy of protein side chain and backbone parameters from ff99SB. *Journal of Chemical Theory and Computation*, 11(8):3696–3713, 2015. doi: 10.1021/acs.jctc.5b00255.

[29] Lee-Ping Wang, Todd J. Martinez, and Vijay S. Pande. Building force fields: An automatic, systematic, and reproducible approach. *The Journal of Physical Chemistry Letters*, 5(11):1885–1891, 2014. doi: 10.1021/jz500737m.

[30] Giulia Crocioni, Dani L. Bodor, Coos Baakman, Farzaneh M. Parizi, Daniel-T. Rademaker, Gayatri Ramakrishnan, Sven A. van der Burg, Dario F. Marzella, João M. C. Teixeira, and Li C. Xue. DeepRank2: Mining 3D protein structures with geometric deep learning. *Journal of Open Source Software*, 9(94):5983, 2024. doi: 10.21105/joss.05983.

[31] Robert T. McGibbon, Kyle A. Beauchamp, Matthew P. Harrigan, Christoph Klein, Jason M. Swails, Carlos X. Hernández, Christian R. Schwantes, Lee-Ping Wang, Thomas J. Lane, and Vijay S. Pande. MDTraj: A modern open library for the analysis of molecular dynamics trajectories. *Biophysical Journal*, 109(8):1528–1532, 2015. doi: 10.1016/j.bpj.2015.08.015.

[32] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. arXiv preprint arXiv:1903.02428, 2019. URL `https://arxiv.org/abs/1903.02428`.

[33] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019. URL `https://openreview.net/forum?id=ryGs6iA5Km`.

[34] Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. Strategies for pre-training graph neural networks. In *International Conference on Learning Representations*, 2020. URL `https://openreview.net/forum?id=HJlWWJSFDH`.

[35] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python.

Journal of Machine Learning Research, 12:2825–2830, 2011. URL `https://jmlr.org/papers/v12/pedregosa11a.html`.

[36] Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, pages 1137–1143, 1995. URL `https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf`.

[37] Mike Schuster and Kuldip K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997. doi: 10.1109/78.650093.

[38] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734, 2014. doi: 10.3115/v1/D14-1179.